Your FastAPI Todos App, Explained Simply

What every file does, how a request travels through it, and what is still unfinished

Verdict first
This is a working skeleton of a to-do API with sign-up and login. Login works and hands out tokens. But the to-do routes **do not check the token yet**, so right now anyone can read, edit or delete anyone's todos. Protecting them with `get_current_user` is the usual next step. Full list of gaps is in section 10.

The 30-second version

- It is a **backend only**. No screens, no buttons. Other programs (the built-in /docs page, Postman, a React app) talk to it using URLs.
- Five Python files, each with **one job**: connect to the database, describe the tables, start the app, handle sign-up/login, handle todos.
- All data lives in one file, **todosapp.db** (SQLite). No database server to install.
- A user **signs up**, **logs in**, receives a **token** (like a wristband at a concert) and shows it on later requests to prove who they are.

Section	What you will learn
1. The big picture	The restaurant picture, and what GET / POST / PUT / DELETE mean
2. Map of the files	Who imports whom, and which file does what
3. database.py	How the app connects to the database
4. models.py	How Python classes become database tables
5. main.py	How the app starts and plugs the pieces together
6. get_db and Depends	The most confusing trick in FastAPI, made simple
7. auth.py	Sign-up, password hashing, login, tokens
8. todos.py	The five to-do endpoints and the checks on incoming data
9. Run it yourself	Folder layout, commands, and a click-by-click test
10. Gaps and next steps	What is unfinished or risky in this code
11. Cheat sheet	Every new word in one page

1. The big picture

Think of a restaurant. You (the **client**) sit at a table and place an order. The **waiter** takes it to the **kitchen**, which grabs ingredients from the **storeroom**, cooks, and sends food back. You never walk into the storeroom yourself.



The customer never walks into the storeroom. Only the waiter does.

Figure 1: the same thing as your app. Request = the order, response = the food.

Your FastAPI app is the waiter and the kitchen. It receives a **request** (a URL plus a method), decides what to do, talks to the database if needed, and sends back a **response** (data plus a status code).

The four methods you will see everywhere

Method	Plain meaning	Restaurant version	In your app
GET	Show me something	"What is on the menu?"	GET / and GET /todo/3
POST	Make a new one	"I would like to order this"	POST /todo and POST /auth/
PUT	Replace it with new details	"Change my order to this"	PUT /todo/3
DELETE	Throw it away	"Cancel my order"	DELETE /todo/3

Together these four are called **CRUD** (Create, Read, Update, Delete). Almost every app you will ever build is CRUD at heart, plus login on top.

2. Map of the files

Read the arrows as "imports from". The top file is the one that starts everything. The bottom is the real data on disk.

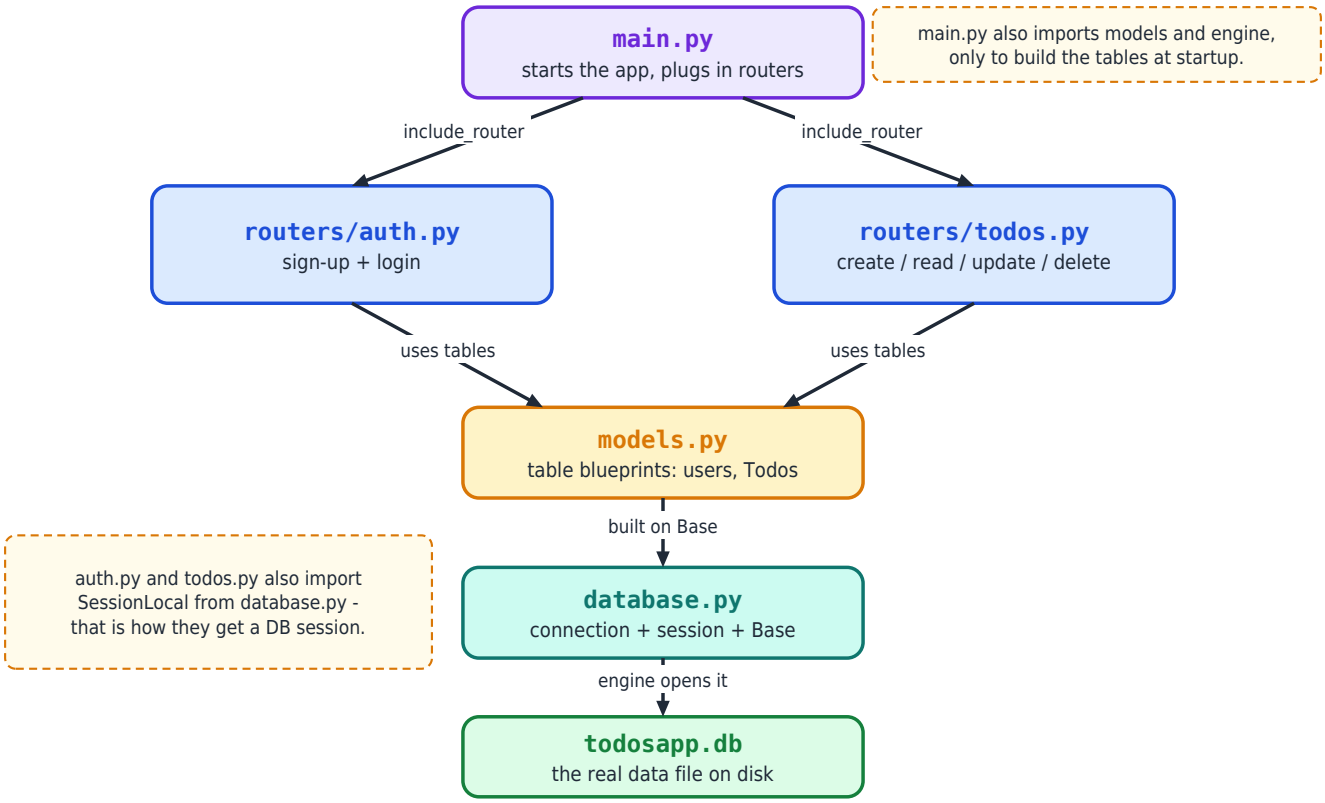


Figure 2: how your files depend on each other.

File	Its one job	Restaurant version
`main.py`	Creates the app, creates the tables, plugs in the two routers	Opens the restaurant and hangs up the menu
`database.py`	Sets up the connection to todosapp.db	The door to the storeroom
`models.py`	Describes the tables: users and todos	Blueprint of the storeroom shelves
`routers/auth.py`	Sign-up, login, and checking tokens	Security desk at the entrance
`routers/todos.py`	Create, read, update, delete todos	The ordering counter

Folder note
Your upload had every file side by side, but `main.py` says `from routers import auth, todos`. That means `auth.py` and `todos.py` must sit inside a folder called `routers`. Section 9 shows the exact layout.

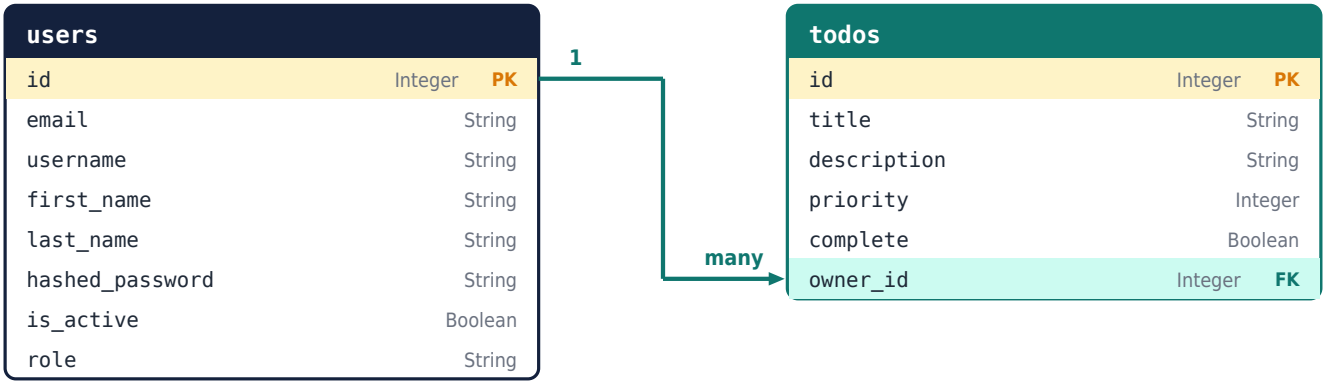
3. database.py: the connection

This file runs no logic. It only builds three tools that the other files borrow. Picture a phone system: the **engine** is the phone line, **SessionLocal** is a machine that hands you a phone, and each **session** is one phone call.

Code (from your file)	What it means in plain words
`SQLALCHEMY_DATABASE_URL = ( 'sqlite:///./todosapp.db')`	The **address** of the database. `sqlite` = type of database. `./todosapp.db` = a file in the current folder (the exact file you uploaded). If the file is missing, SQLite creates it.
`engine = create_engine( SQLALCHEMY_DATABASE_URL, connect_args={ 'check_same_thread': False})`	The **phone line** to the database, made once. The `check_same_thread` part: SQLite normally refuses to be used by any thread other than the one that opened it. FastAPI handles requests on different threads, so we switch that rule off.
`SessionLocal = sessionmaker( autocommit=False, autoflush=False, bind=engine)`	A **factory** that makes sessions. A session = one conversation with the database. `autocommit=False`: nothing is saved until you write `db.commit()`. `autoflush=False`: SQLAlchemy will not quietly send half-finished changes.
`Base = declarative_base()`	An empty **parent class**. Any class that inherits from `Base` (in models.py) becomes a table. This is how SQLAlchemy keeps a list of all your tables.

4. models.py: tables written as classes

Here is the core idea: **one class = one table**, **one `Column` = one column**, and later **one object = one row**. So `Todos(title='Buy milk', priority=2)` in Python is one row in the todos table.



One user can own many todos. owner_id stores the id of that user.

Figure 3: your two tables. PK = primary key (the unique ID). FK = foreign key (a link to another table).

Code (from your file)	What it means in plain words
`class users(Base): __tablename__ = 'users'`	Inheriting from `Base` makes this a table. `__tablename__` is the name inside the database file.
`id = Column(Integer, primary_key=True, index=True)`	A unique number for every row (1, 2, 3...), filled in automatically. `index=True` makes searching by id fast, like the index at the back of a book.
`email = Column(String, unique=True)` `username = Column(String, unique=True)`	`unique=True`: the database refuses a second user with the same email or username.
`first_name = Column(String)` `last_name = Column(String)`	Plain text columns.

Code (from your file)	What it means in plain words
`hashed_password = Column(String)`	Not the password itself. A scrambled version of it (section 7).
`is_active = Column(Boolean, default=False)` `role = Column(String)`	A True/False flag, and a text label like "admin" or "user". Sign-up sets `is_active=True`, so the default is never used there.

`class Todos(Base):`
`__tablename__ = 'todos'`
`title / description = Column(String)`
`priority = Column(Integer)`
`complete = Column(Boolean, default=False)`
	The to-do table. Text for title and description, a number for priority, a True/False for done.

`owner_id = Column(Integer, ForeignKey(users.id))`	**The link between the tables.** It stores the id of the user who owns the todo. (SQLite does not police this by default, but it is the declared relationship.)

Do not mix up the two kinds of "model"

	Where	What it describes	Used for
SQLAlchemy model	`models.py`: users, Todos	A table in the **database**	Saving and fetching rows
Pydantic model	`CreateUserRequest`, `TodoRequest`, `Token`	The shape of data coming in or going out over the **internet**	Checking input before you trust it

5. main.py: starting the app

Code (from your file)	What it means in plain words
`from routers import auth, todos`	Brings in the two route files from the `routers` folder.
`app = FastAPI()`	Creates the app. This is the object that `uvicorn main:app` runs.
`models.Base.metadata.create_all(bind=engine)`	"Look at every class that inherits from Base, and **create its table if it does not exist yet**." This is why todosapp.db has tables without you writing any SQL. **Catch:** it never changes an existing table, so if you add a column later it will not appear (tools called migrations solve that, later).
`app.include_router(auth.router)` `app.include_router(todos.router)`	Plugs both route files into the app. Without these two lines, their URLs would not exist. The auth router has `prefix='/auth'`, so its URLs become `/auth/` and `/auth/token`. The todos router has no prefix, so you get `/` and `/todo`.

Why routers at all? If every endpoint lived in main.py, it would turn into one giant file. A router is a mini-app: you write routes in a separate file, then plug it in. You even wrote this as a comment at the top of auth.py.

6. get_db and Depends: the trick you will see in every file

Every endpoint that touches the database needs a session. Writing "open session, use it, close it" in every function would be repetitive and easy to forget. So you write it once, and tell FastAPI: "*before running my function, run this first and hand me the result.*" That instruction is `Depends`.

Code (from your file)	What it means in plain words

`def get_db():`
`  db = SessionLocal()`
`  try:`
`  yield db`
`  finally:`
`  db.close()`
	Opens a session, **lends it** to the endpoint with `yield`, and when the endpoint is done, jumps back and closes it. `finally` guarantees the close happens even if the endpoint crashes. Never leaving sessions open matters; they are like unclosed phone calls.

`db_dependency = Annotated[` `  Session, Depends(get_db)]`	Just a **nickname**. It means "a Session, and FastAPI should get it by running get_db".
`async def read_all(db: db_dependency):` `  return db.query(Todos).all()`	Because of the nickname, `db: db_dependency` is all you write. FastAPI runs get_db for you and passes the session in as `db`.

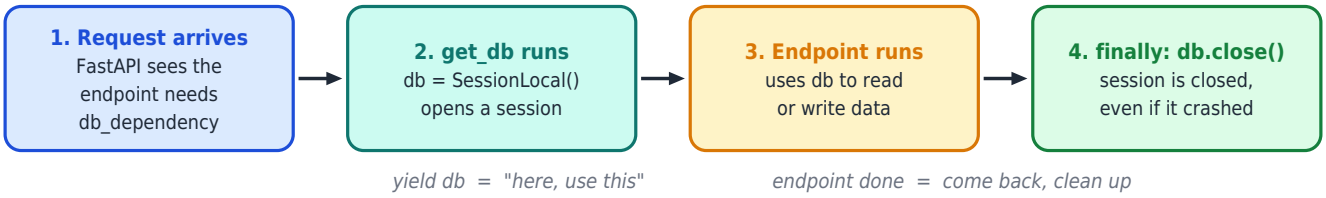


Figure 4: the life of one session. It exists only while one request is being handled.

> **Why yield and not return?**
> `return` ends a function. `yield` **pauses** it. FastAPI pauses get_db at yield, runs your endpoint, then resumes get_db so the cleanup after yield runs. That pause-and-resume is the whole trick.

Notice that `get_db` and `db_dependency` are copy-pasted in both auth.py and todos.py. It works, but later you would move them into one shared place.

7. auth.py: sign-up, login and tokens

This file answers two questions: **"Who are you?"** (login) and **"Can you prove it on later requests?"** (token). Code below is shortened slightly to fit the page; nothing about how it works is changed.

7.1 The setup lines

Code (from your file)	What it means in plain words

`router = APIRouter(`
`  prefix='/auth',`
`  tags=['auth'])`
	A mini-app. `prefix`: every URL here starts with /auth. `tags`: groups these endpoints under "auth" on the /docs page.

`SECRET_KEY = '3869...4829'` `ALGORITHM = 'HS256'`	The secret used to sign tokens, and the signing method. **Keep the key private** (see section 10, gap 1).

`bcrypt_context = CryptContext(`
`  schemes=['bcrypt'],`
`  deprecated='auto')`
	The **password scrambler**. `.hash(pw)` scrambles a password. `.verify(pw, hash)` checks a typed password against the stored scramble.

`oauth2_bearer = OAuth2PasswordBearer(` `  tokenUrl='auth/token')`	Tells FastAPI: "to be logged in, send a token in the `Authorization` header", and "tokens come from /auth/token". This is also what creates the **Authorize** button on /docs.

`class CreateUserRequest(BaseModel):`
`  username: str`
`  email: str`
`  first_name: str`
`  last_name: str`
`  password: str`
`  role: str`
	The **form** a new user must fill in. If the JSON is missing a field or has the wrong type, FastAPI rejects it automatically with a 422 error. Note it has `password` but the table has `hashed_password`: you never store the raw one.

`class Token(BaseModel):`
`  access_token: str`
`  token_type: str`
	The shape of the **reply** after login. Used as `response_model` below.

7.2 Why passwords are hashed

If you stored "mypassword123" and someone stole your database, they would have everyone's real passwords. A **hash** is a one-way scramble: easy to make, practically impossible to reverse. At login you scramble what the user typed and compare the two scrambles. You never need the original, so you never keep it.

7.3 Sign-up: POST /auth/

Code (from your file)	What it means in plain words

`@router.post("/", status_code=201)`
`async def create_user(`
`  db: db_dependency,`
`  create_user_request: CreateUserRequest):`
`  create_user_model = users(`
`  email=create_user_request.email,`
`  ...`
`  hashed_password=bcrypt_context`
`  .hash(create_user_request.password),`
`  is_active=True)`
`  db.add(create_user_model)`
`  db.commit()`
	Step by step:
	1. FastAPI checks the incoming JSON against `CreateUserRequest`.
	2. It opens a session for you (`db`).
	3. You build a `users` row object from the fields, scrambling the password on the way.
	4. `db.add` puts it in the basket. `db.commit` actually saves it.
	5. Status 201 = "Created". Nothing is returned.

7.4 Login: POST /auth/token

Code (from your file)	What it means in plain words

`def authenticate_user(username, password, db):`
`  user = db.query(users).filter(`
`  users.username == username).first()`
`  if not user:`
`  return False`
`  if not bcrypt_context.verify(`
`  password, user.hashed_password):`
`  return False`
`  return user`
	Finds the user by username, then checks the typed password against the stored scramble. Returns the user if both pass, otherwise `False`. It gives the same failure for "no such user" and "wrong password" so attackers cannot tell which one it was.

`@router.post("/token", response_model=Token)`
`async def login_for_access_token(`
`  form_data: Annotated[`
`  OAuth2PasswordRequestForm, Depends()],`
`  db: db_dependency):`
`  user = authenticate_user(`
`  form_data.username,`
`  form_data.password, db)`
`  if not user:`
`  raise HTTPException(401, ...)`
`  token = create_access_token(`
`  user.username, user.id,`
`  timedelta(minutes=20))`
`  return {"access_token": token,`
`  "token_type": "bearer"}`
	The login endpoint. `OAuth2PasswordRequestForm` reads `username` and `password` from a **form** (not JSON). That is what the Authorize button on /docs sends. If the check fails, 401 "Unauthorized". If it passes, you build a token that lives for 20 minutes and send it back.

7.5 What a token (JWT) really is

After login, the user should not need to send their password with every request. Instead the server gives them a **token**, like a **wristband at a concert**: you show your ticket once at the gate, get a wristband, and from then on you just show the wristband. A JWT has three parts joined by dots.

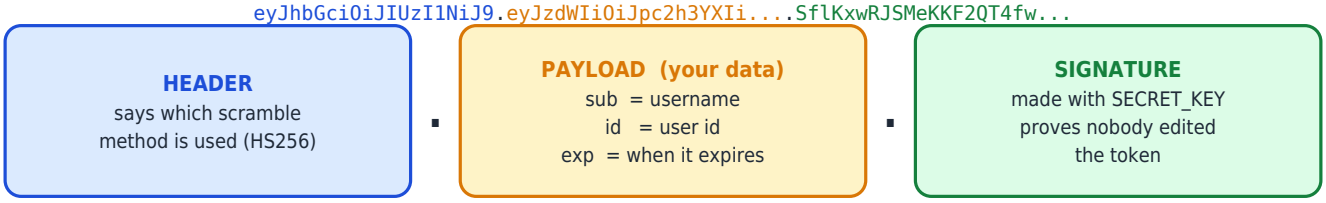


Figure 5: the three parts of a JWT. The payload is only encoded, not secret, so never put passwords inside it.

Code (from your file)	What it means in plain words

`def create_access_token(username, user_id,`
`  expires_delta):`
`  encode = {'sub': username,`
`  'id': user_id}`
`  expires = datetime.now(timezone.utc)`
`  + expires_delta`
`  encode.update({'exp': expires})`
`  return jwt.encode(encode, SECRET_KEY,`
`  algorithm=ALGORITHM)`
	Builds the payload (`sub` = who, `id` = user id, `exp` = expiry time), then `jwt.encode` signs it with `SECRET_KEY`. If anyone edits the payload, the signature no longer matches, and the token is rejected. That is why the server can trust it without asking the database.

`async def get_current_user(`
`  token: Annotated[str,`
`  Depends(oauth2_bearer)]):`
`  try:`
`  payload = jwt.decode(token,`
`  SECRET_KEY, algorithms=[ALGORITHM])`
`  username = payload.get('sub')`
`  user_id = payload.get('id')`
`  if username is None or user_id is None:`
`  raise HTTPException(401, ...)`
`  return {"Username": username,`
`  "ID": user_id}`
`  except:`
`  raise HTTPException(401, ...)`
	The **bouncer**. It pulls the token out of the `Authorization: Bearer ...` header, checks the signature and the expiry, and returns who the user is. Bad or expired token = 401. **Important:** no endpoint calls this yet.

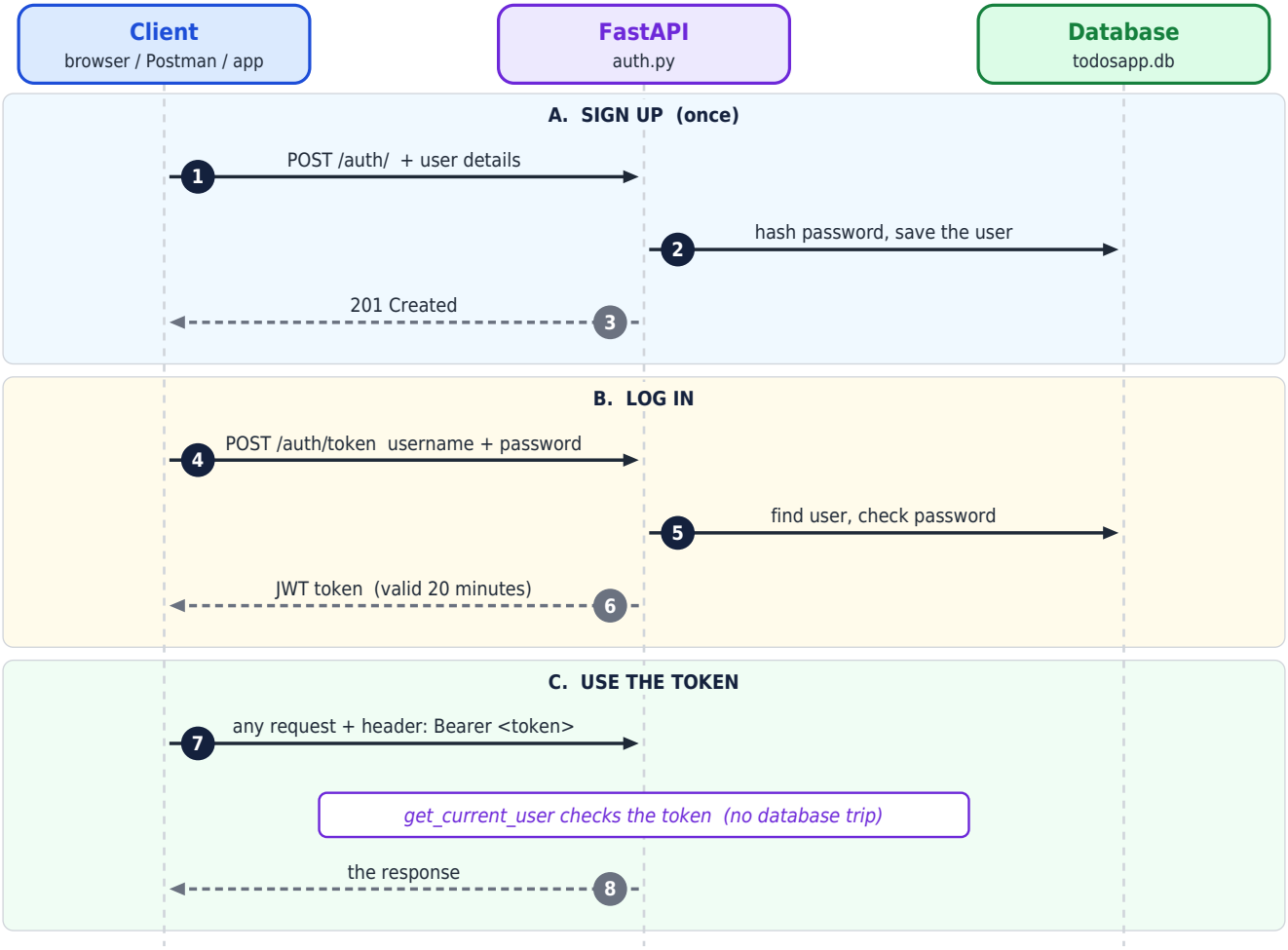


Figure 6: the full login story, in order. Solid arrows are requests, dashed arrows are replies.

8. todos.py: the to-do endpoints

Same structure as auth.py: a router, the same `get_db`, and one Pydantic class that validates incoming data.

Code (from your file)	What it means in plain words

`class TodoRequest(BaseModel):`
`  title: str = Field(min_length=3)`
`  description: str = Field(`
`  min_length=3, max_length=100)`
`  priority: int = Field(gt=0, lt=6)`
`  complete: bool`
	The rules for creating or updating a todo. Title at least 3 characters. Description 3 to 100 characters. Priority greater than 0 and less than 6, so **1 to 5**. `complete` must be true or false. Break a rule and FastAPI replies 422 with a message saying which one.

`todo_id: int = Path(gt=0)`	The id in the URL must be a whole number above 0. `/todo/abc` or `/todo/-5` are rejected before your code runs.

Method + URL	What it does	Success	Can fail with
`GET /`	List all todos	200	-
`GET /todo/{todo_id}`	Get one todo by id	200	404 not found, 422 bad id
`POST /todo`	Create a todo from the JSON body	201	422 invalid data
`PUT /todo/{todo_id}`	Replace title, description, priority, complete	204 (empty body)	404, 422
`DELETE /todo/{todo_id}`	Remove the todo	204 (empty body)	404, 422

Code (from your file)	What it means in plain words
`db.query(Todos).all()`	Fetch every row of the todos table. Same idea as SQL: `SELECT * FROM todos`.
`db.query(Todos).filter(` `  Todos.id == todo_id).first()`	Find the todo with that id and give me the first match, or `None` if there is none. `None` is why the code then raises 404. SQL: `SELECT * FROM todos WHERE id = ? LIMIT 1`.
`todo_model = Todos(` `  **todo_request.model_dump())`	`model_dump()` turns the checked request into a dictionary. The `**` unpacks it into `title=..., description=..., ...`. Result: a new row object, ready to save.

`todo_model.title = todo_request.title`
`...`
`db.add(todo_model)`
`db.commit()`
	Update: fetch the row, overwrite its fields, save. SQLAlchemy notices the changes and runs an UPDATE on commit.

`db.query(Todos).filter(`
`  Todos.id == todo_id).delete()`
`db.commit()`
	Delete the matching row, then commit to make it permanent.

One request, start to finish: POST /todo

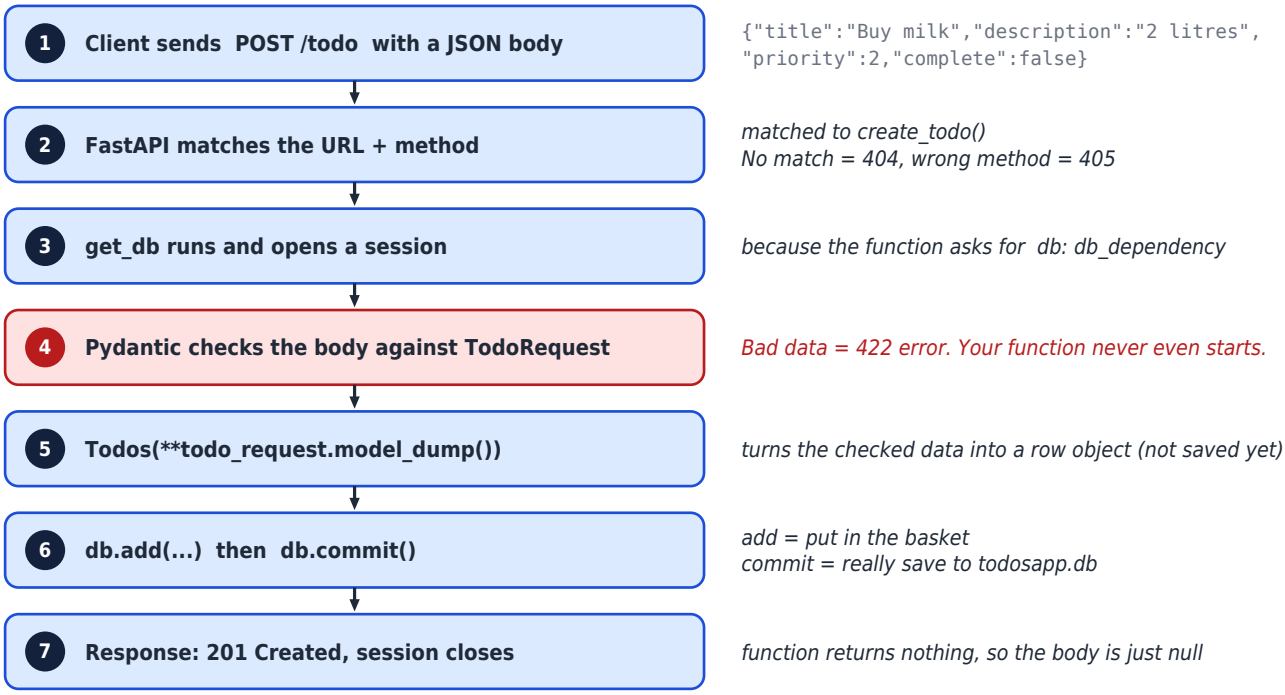


Figure 7: what happens, in order, when someone creates a todo.

Status codes in one glance

Code	Meaning	Seen when
200	OK	Reading data worked
201	Created	A new todo or user was saved
204	No Content	Update or delete worked, nothing to send back
401	Unauthorized	Wrong login, or bad / expired token
404	Not Found	No todo with that id
422	Unprocessable	Incoming data broke a rule (Pydantic said no)

9. Run it yourself

Folder layout it expects

```
your-project/
├── main.py
├── database.py
├── models.py
├── todosapp.db        (created automatically)
└── routers/
    ├── auth.py
    └── todos.py
```

Commands

```
pip install fastapi uvicorn sqlalchemy "passlib[bcrypt]" python-jose python-multipart
uvicorn main:app --reload
```

- Run `uvicorn` from the folder that contains main.py, otherwise lines like `from database import ...` fail.
- `python-multipart` is needed for the login form (`OAuth2PasswordRequestForm`).
- If sign-up or login throws a strange bcrypt error, try `pip install bcrypt==4.0.1`. Newer bcrypt versions can clash with passlib.

Click-by-click test on the /docs page

1. Open `http://127.0.0.1:8000/docs`. FastAPI built this page for you from your code. You can send real requests from it.
2. Try **POST /auth/**. Fill in a user (any role text) and Execute. Expect 201.
3. Click **Authorize** (top right), enter that username and password. Behind the scenes it calls /auth/token and keeps the token.
4. Try **POST /todo** with a valid body (priority 1 to 5), then **GET /** to see it. Then try PUT and DELETE on its id.
5. Now break a rule on purpose (priority 9, title "ab"). Read the 422 message. That is Pydantic at work.

10. Gaps and next steps (honest list)

Most of this is normal for a course project at this stage. Gaps 1 and 2 matter most.

#	Gap	Why it matters	Fix
1	**SECRET_KEY is typed into auth.py**	Anyone who sees your code (GitHub!) can forge a token for any user.	Move it to an environment variable or a `.env` file, add `.env` to `.gitignore`, and generate a fresh key (`openssl rand -hex 32`), because the current one has now been in code files.
2	**Todo routes are not protected**	No route uses get_current_user, so no login is needed. `owner_id` is never filled in, so it stays empty (NULL), and everyone sees everyone's todos.	Add a user dependency to each route, set `owner_id` from the token when creating, and filter queries by `owner_id`.
3	Anyone can pick any role; no duplicate check	A user can sign up as "admin". A duplicate username or email hits the unique rule and returns a 500 error.	Limit allowed roles, check for existing users first, and return a clear 400 message.
4	Bare `except:` in get_current_user	It catches every error, including your own HTTPException, and can hide real bugs. `JWTError` is imported but never used.	Write `except JWTError:` instead.
5	get_db copied into two files	Fix a bug in one, forget the other.	Move it to one shared file and import it.

#	Gap	Why it matters	Fix
6	create_all does not update old tables	Add a column later and the existing database will not get it.	For now delete todosapp.db while learning; later learn Alembic migrations.
7	Old import in database.py	`sqlalchemy.ext.declarative` is the old home of declarative_base. It still works but gives a deprecation warning on SQLAlchemy 2.x.	Use `from sqlalchemy.orm import declarative_base`.

11. Cheat sheet: every new word

Word	In plain words
Endpoint	One URL + method that does one thing, e.g. `GET /todo/3`. In code: a function with `@router.get(...)` on top.
Router	A mini-app in its own file that you plug into the main app.
Request / Response	What the client sends / what the server sends back.
JSON	Text format for data, like `{"title": "Buy milk"}`. The language clients and servers use to talk.
Pydantic model	A class that describes and *checks* incoming data (`TodoRequest`).
SQLAlchemy model	A class that describes a database table (`users`, `Todos`).
Engine	The connection to the database file.
Session	One conversation with the database. Open, use, commit, close.
commit	Make the changes permanent. Without it, nothing is saved.
Dependency (Depends)	Something FastAPI runs *for* your function and hands to it as a parameter.
Hash	A one-way scramble. Used so passwords are never stored in readable form.
JWT / token	A signed wristband the server gives you at login. You show it on later requests.
Bearer	The word in the header: `Authorization: Bearer <token>`. It means "whoever bears (carries) this token".
Swagger / /docs	The test page FastAPI generates for you automatically.
uvicorn	The server program that actually runs your app and listens for requests.